

Docker: Comprehensive Theory and Concepts Study Guide

Module 1: Docker Basics

- **What is Docker?** Docker is an open-source platform that uses containerization technology to package application code, runtimes, system tools, and libraries into a single lightweight, portable container that runs consistently in any environment.
 - **Why Docker?**
 - **Portability:** "Build once, run anywhere." Eliminates the "it works on my machine" problem.
 - **Isolation:** Sandbox applications from one another, preventing version conflicts on host machines.
 - **Resource Efficiency:** Containers consume significantly less CPU and RAM than virtual machines.
 - **Consistency:** Identical environments across development, staging, and production.
 - **Virtual Machine vs. Docker**
 - **Virtual Machine (VM):** Hypervisor-based. Each VM includes a complete guest operating system, virtual drivers, and binaries, causing high resource overhead and slow startup times.
 - **Docker Container:** OS-level virtualization. Containers share the host operating system kernel, separating user spaces via Linux namespaces and cgroups. They are extremely lightweight, start in seconds, and consume minimal RAM.
 - **Docker Architecture** Follows a client-server architecture:
 - **Docker Client (CLI):** The interface users use to interact with Docker (e.g. `docker build`, `docker run`).
 - **Docker Daemon (dockerd):** The background process running on the host that listens for API requests and manages Docker objects (images, containers, networks, volumes).
 - **Docker Registry:** Storage repository for images (e.g., Docker Hub, Amazon ECR).
 - **OCI (Open Container Initiative):** A governance body that establishes industry standards for container formats and runtime technologies, ensuring cross-compatibility.
-

Module 2: Images & Containers

- **Docker Image** An immutable, read-only template containing instructions to build a container. Formed of stacked, read-only layers representing filesystem changes.
 - **Docker Container** A running, writeable instance of an image. When a container starts, Docker adds a thin, writeable layer (the container layer) on top of the read-only image layers. Any modification during runtime is stored in this writeable layer.
 - **Image Layers** Each instruction in a Dockerfile creates a read-only layer. Docker caches these layers during build time to accelerate rebuilds (Layer Caching).
 - **Container Lifecycle & States**
 - *Created*: Container exists but is not running.
 - *Running*: Active container executing its primary process.
 - *Paused*: Processes are suspended.
 - *Stopped*: Primary process has exited.
 - *Deleted*: Container configuration and writeable layer are removed.
 - **Docker Registry Commands**
 - `docker pull <image>` : Downloads an image from a registry.
 - `docker tag <source_image> <target_image>:<tag>` : Renames/tags an image.
 - `docker push <target_image>:<tag>` : Uploads an image to a registry.
-

Module 3: Dockerfile Directives

A text file containing the sequential instructions called to compile a Docker image.

- `FROM` : Defines the base image (e.g., `FROM python:3.11-slim`).
- `RUN` : Executes commands inside a new layer during the build phase (e.g., `RUN pip install -r requirements.txt`).
- `COPY` : Copies files/folders from the host system filesystem to the container filesystem.
- `ADD` : Similar to `COPY`, but can download files from URLs and automatically extracts compressed archives (`tar.gz`). Prefer `COPY` unless extraction is needed.
- `WORKDIR` : Sets the working directory inside the container for subsequent directives.
- `ENV` : Sets persistent environment variables.
- `EXPOSE` : Informs Docker that the container listens on specified ports at runtime (acts as metadata documentation; does not publish ports to the host).
- `USER` : Sets the UID or username to run subsequent instructions, improving security by avoiding root privileges.
- `HEALTHCHECK` : Tells Docker how to test the container to check if it is still working.
- `CMD` VS. `ENTRYPOINT`

- `ENTRYPOINT` : Configures the container to run as an executable. Arguments cannot be overridden easily during launch.
 - `CMD` : Sets default arguments for the `ENTRYPOINT` . Can be easily overridden by passing parameters at the end of the `docker run` command.
-

Module 4: Storage Options

By default, files created inside containers are ephemeral, written to the temporary container layer, and lost when the container is deleted. Docker provides options to persist data:

- **Volumes** Stored in a part of the host filesystem managed entirely by Docker (`/var/lib/docker/volumes/` on Linux). The preferred way to persist database data. Decoupled from host OS path structures.
 - *Named Volumes*: Explicitly named, reusable across containers (e.g., `docker run -v my_db_data:/var/lib/mysql`).
 - *Anonymous Volumes*: Automatically created without names, deleted when the container is removed with the `-v` flag.
 - **Bind Mounts** Maps a specific, absolute path directory on the host machine directly to a directory inside the container (e.g., `-v /home/user/app:/app`). Used in development for live-reloading code.
 - **tmpfs Mount** Stores data temporarily in the host system's memory (RAM) only. Never written to the host filesystem.
-

Module 5: Networking

Docker uses drivers to isolate and connect containers:

- **Bridge (Default)**: Creates a private virtual network internal to the host. Containers on the same bridge network can communicate using IP addresses or container names (automatic DNS service discovery).
 - **Host**: Removes network isolation. The container shares the host network interface directly (e.g., a container on port 80 maps directly to host port 80).
 - **None**: Disables all networking interfaces for the container.
 - **Overlay**: Connects multiple Docker daemons across different physical hosts (Swarm/Kubernetes), allowing containers to communicate across hosts.
 - **Port Mapping**: Exposing a container port to the outside host network using the `-p <host_port>:<container_port>` flag.
-

Module 6: Docker Compose

- **What is Docker Compose?** A tool used to define, configure, and run multi-container applications using a single YAML configuration file (`docker-compose.yml`).
 - **Key Concepts**
 - **Services:** Defines the containers to build and run.
 - **Networks:** Configures shared virtual networks to connect services.
 - **Volumes:** Configures shared persistent storage.
 - *Example `docker-compose.yml`:* `yml version: '3.8' services: web: build: . ports: - "8000:8000" depends_on: - db db: image: postgres:15 volumes: - pgdata:/var/lib/postgresql/data volumes: pgdata:`
-

Module 7: Image Optimization

- **Multi-stage Builds** A pattern that uses multiple `FROM` instructions in a single Dockerfile. You build artifacts in early stages using heavy dependencies (compilers, SDKs) and copy only the final compiled outputs to a lightweight runtime base image. Reduces final image sizes by up to 90%.
 - **Optimization Best Practices**
 - **Layer Caching:** Order instructions from least-frequently changed to most-frequently changed to maximize caching (e.g., copy `requirements.txt` and run `pip install` before copying the source code).
 - **.dockerignore:** A text file listing files and directories (like `.git`, `node_modules`, `venv`) that should be ignored during `COPY` operations, preventing build bloat.
 - **Alpine Images:** Using lightweight Linux base images (Alpine or slim) to reduce security vulnerabilities and disk footprint.
-

Module 8: Security

- **Rootless Docker:** Running the Docker daemon and containers as a non-root user to mitigate privilege escalation attacks.
- **Image Scanning:** Analyzing container images for known security vulnerabilities using scanning tools (e.g. *Docker Scout* or *Trivy*).
- **Secrets Management:** Never write raw API keys or passwords inside Dockerfiles or image layers. Pass secrets at runtime using environment variables, external secret stores, or Docker Compose secrets.
- **Distroless Images:** Base images containing only your application and its runtime dependencies. They do not contain package managers, shells, or standard Linux utilities, reducing attack surfaces.

Module 9: Production & Orchestration

- **Resource Limits:** Restricting CPU and RAM usage to prevent a single container from consuming all host resources (e.g., `--memory="512m"` `--cpus="1.5"`).
 - **Restart Policies:** Tells Docker how to handle container exits (e.g., `no` , `always` , `on-failure` , `unless-stopped`).
 - **Orchestration**
 - **Docker Swarm:** Docker's built-in clustering tool. Easy to configure, best for small projects.
 - **Kubernetes (K8s):** An enterprise-grade container orchestration system designed to automate deployment, scaling, and management of containerized applications across massive clusters.
 - **Troubleshooting Commands**
 - `docker logs <id>` : Displays container logs.
 - `docker inspect <id>` : Returns detailed system-level configuration metadata.
 - `docker exec -it <id> /bin/bash` : Opens an interactive shell inside a running container.
 - `docker stats` : Displays live resource usage statistics (CPU, memory, network).
 - `docker system prune` : Deletes all unused containers, networks, and dangling build caches to free disk space.
-

★ High-Yield Interview Questions (Docker)

1. **What is Docker and how does it differ from a Virtual Machine?** Docker is an OS-level virtualization platform that packages apps into containers. Unlike VMs, which use hypervisors and include complete guest operating systems, Docker containers share the host kernel via namespaces and cgroups, making them much faster and lighter.
2. **Explain the difference between a Docker Image and a Container.** An image is an immutable, read-only template consisting of stacked layers. A container is a running, writeable instance of that image with a thin writeable layer added on top.
3. **CMD vs. ENTRYPOINT in a Dockerfile?** `ENTRYPOINT` defines the default executable command run when the container starts. `CMD` provides default arguments for that command. `CMD` can be easily overridden during container startup; `ENTRYPOINT` requires explicit CLI flags to override.
4. **What is a Multi-stage build and why is it useful?** It is a method of using multiple `FROM` stages in a Dockerfile. You compile artifacts in early stages using build tools, then copy only the final compiled outputs to a lightweight runtime image, minimizing final image sizes.
5. **How do you persist data in Docker?** Using **Volumes** (managed by Docker in a dedicated directory) or **Bind Mounts** (mapping a specific directory on the host to the container).

Volumes are preferred for production.

6. What are the different Docker network drivers?

- *Bridge (Default)*: Private virtual network on a single host.
- *Host*: Shares the host network interface directly.
- *None*: Disables networking.
- *Overlay*: Connects containers across multiple physical hosts.

7. How do you optimize layer caching during builds? Order commands in the Dockerfile from least-frequently changed to most-frequently changed (e.g., copying package manifests and installing dependencies before copying source code).

8. What is the purpose of a `.dockerignore` file? It prevents large or sensitive files (like `.git`, test directories, and local dependencies) from being copied into the build context, reducing build times and image sizes.

9. How do you run a multi-container application? By defining all containers, networks, and volumes in a `docker-compose.yml` file and executing `docker-compose up`.

10. How do you limit a container's memory and CPU usage? By passing resource limit flags during runtime (e.g., `docker run -m 512m --cpus 1.5 <image>`) or defining `resources` limits inside a Docker Compose file.



Module Summary & Key Takeaways

Theory Focus: Virtualization vs Containerization, Dockerfile Instructions, Storage Mounts, and Compose.

This module examines Docker container patterns and microservice environment management. Key takeaways include:

- **Container Isolation:** Docker isolates applications using host kernel namespaces and control groups (cgroups), creating lightweight containers that share the host OS, unlike virtual machines which require heavy guest operating systems.
- **Image Structuring & Layer Caching:** Docker builds images as stackable, read-only layers defined in a Dockerfile. Images are optimized by using small base configurations (slim/Alpine), running multi-stage builds, and arranging directives to maximize layer reuse.
- **Networking & Volume Persistence:** Transient writeable container layers discard data upon termination. Persistence is achieved via managed Volumes or host-mapped Bind Mounts, communicating securely over isolated bridge, host, or overlay network configurations.